

Modificación del planificador de CPU y la gestión de memoria en xv6

Scheduler por prioridades con Aging y Fork con Copy-on-Write



Autor	Omar Esteban Agredo
Asignatura	Sistemas Operativos
Profesor	Jefferson Amado Peña Torres
Repositorio	github.com/omar-agredo/xv6-scheduler-cow-project
Video	https://youtu.be/xQVmq0OP8is

Resumen

Este informe documenta el análisis, diseño, implementación y validación de dos modificaciones sobre xv6-riscv: un planificador por prioridades con mecanismo de aging y una optimización de fork mediante Copy-on-Write (COW). La primera modificación incorpora prioridades base entre 0 y 4, una llamada al sistema para configurarlas y un cálculo de prioridad efectiva que mejora con el tiempo de espera en estado RUNNABLE. La segunda sustituye la copia inmediata de páginas físicas durante fork por páginas compartidas de solo lectura, un bit PTE_COW, contadores de referencia y resolución bajo demanda de los fallos de escritura.

El trabajo se desarrolló en Fedora 44 con QEMU para RISC-V y toolchain riscv64-linux-gnu. La validación combinó una línea base, cargas específicas para prioridades y aging, una prueba de aislamiento COW con padre, hijo y nieto, ejecución con una y varias CPU virtuales, y la suite general usertests -q. El proceso de prioridad 4 reanudó después de 81 ticks, coherente con los 80 ticks teóricos requeridos para ascender cuatro niveles. La prueba COW confirmó la independencia de 16 páginas modificadas por tres procesos y, después de corregir un acceso superior a MAXVA, la suite terminó con ALL TESTS PASSED.

Palabras clave: xv6, RISC-V, scheduler, prioridad, aging, starvation, memoria virtual, Copy-on-Write, page fault, fork.

Contenido

1. Introducción
2. Objetivos
3. Alcance y metodología
4. Entorno de desarrollo
5. Análisis del scheduler original
6. Análisis de la memoria original
7. Diseño del scheduler por prioridades con aging
8. Implementación del scheduler
9. Pruebas y resultados del scheduler
10. Diseño de Copy-on-Write
11. Implementación de Copy-on-Write
12. Resultados y comparación de memoria
13. Pruebas generales y regresiones
14. Problemas encontrados y depuración
15. Ventajas y limitaciones
16. Control de versiones y reproducibilidad
17. Demostración
18. Conclusiones
19. Referencias
- Anexo A. Archivos modificados
- Anexo B. Cobertura de la rúbrica

1. Introducción

Un sistema operativo debe repartir el tiempo de CPU entre procesos y administrar la memoria física de forma segura y eficiente. El planificador determina qué proceso ejecuta cada CPU y cuándo debe cederla; la gestión de memoria traduce direcciones virtuales, controla permisos y decide cuándo asignar o liberar páginas físicas. Ambos subsistemas interactúan con interrupciones, estados de proceso, traps, locks y estructuras compartidas, por lo que modificarlos obliga a razonar sobre concurrencia y corrección, no solo sobre el resultado visible.

xv6 es un sistema operativo docente inspirado en Unix Version 6 y portado a RISC-V para el curso MIT 6.1810. Su código es deliberadamente compacto: permite seguir el flujo desde una llamada al sistema o una interrupción hasta las estructuras internas del kernel sin la complejidad de un sistema de producción. Esta característica lo convierte en una plataforma adecuada para comparar un comportamiento original con una política modificada y para verificar el efecto de decisiones de diseño concretas.

El proyecto se concentró en dos componentes. En planificación se reemplazó la selección uniforme por prioridades base y aging, evitando que una tarea de baja prioridad espere indefinidamente. En memoria se mantuvo el soporte de lazy allocation ya presente en la línea base y se eligió una mejora diferente: Copy-on-Write para fork. La decisión evitó duplicar funcionalidad existente y permitió estudiar permisos de PTE, fallos de página y vida útil de páginas compartidas.

2. Objetivos

2.1 Objetivo general

Analizar y modificar los algoritmos de planificación de CPU y gestión de memoria de xv6-riscv, implementando un scheduler por prioridades con aging y un fork con Copy-on-Write, y evaluar su comportamiento mediante pruebas reproducibles antes y después de los cambios.

2.2 Objetivos específicos

- Configurar un entorno de compilación y emulación RISC-V capaz de ejecutar xv6 de manera estable.
- Identificar estructuras, funciones, locks, transiciones de estado e interrupciones relevantes para el scheduler original.
- Describir el esquema Sv39, el asignador de páginas físicas, el crecimiento de memoria, lazy allocation y la copia original de memoria durante fork.
- Diseñar una política de prioridades con rango acotado y un mecanismo de aging basado en ticks.
- Exponer una interfaz de usuario setpriority() y construir pruebas de prioridades y de prevención de starvation.
- Diseñar Copy-on-Write con un bit de software en el PTE, contadores de referencia y resolución de fallos de escritura.
- Comprobar aislamiento entre procesos, compatibilidad con copyout(), lazy allocation y direcciones inválidas.
- Comparar cualitativa y cuantitativamente la línea base y la versión modificada.
- Mantener un historial de cambios trazable mediante ramas, commits y tags estables.

3. Alcance y metodología

La metodología siguió una secuencia incremental. Primero se compiló y ejecutó una línea base sin cambios funcionales. Después se identificaron los puntos de entrada al estado RUNNABLE, el flujo de interrupción del temporizador y la implementación original de uvmcopy(). Cada modificación se dividió en commits pequeños; después de cada etapa se compiló, se arrancó xv6 y se ejecutó una prueba enfocada. Finalmente se corrió la suite completa de regresión.

1. Preparación del entorno y creación de tags de línea base.

2. Análisis del scheduler y diseño de prioridades con aging.
3. Implementación de metadatos, syscall, selección y pruebas específicas.
4. Análisis de la memoria y confirmación del lazy allocation existente.
5. Implementación incremental de Copy-on-Write y contadores de referencia.
6. Pruebas de aislamiento, copyout, múltiples generaciones y MAXVA.
7. Regresión completa, documentación y publicación del repositorio.

Criterio de veracidad: los tiempos y resultados presentados corresponden a ejecuciones registradas. La comparación de páginas copiadas en el escenario de “4 de 16 páginas modificadas” es un cálculo analítico, no una medición instrumentada del kernel.

4. Entorno de desarrollo

Componente	Configuración utilizada
Sistema anfitrión	Fedora 44, arquitectura x86_64
Kernel anfitrión	Linux 7.1.3
Emulación	QEMU 10.2.2 con máquina virt y CPU RISC-V
Compilador cruzado	riscv64-linux-gnu GCC 16.1.1
Binutils	2.46
GDB	17.2
GNU Make	4.4.1
Git	2.55
Código base	xv6-riscv rev5; tag local baseline-original
Rama final	feature/scheduler-priority-aging
Commit final	f83ee36

La línea base generó kernel/kernel, pero GCC 16 detuvo la construcción de user/usertests.c porque el Makefile usa -Werror y la variable fsblocks producía la advertencia “set but not used”. Se agregó -Wno-error=unused-but-set-variable. El cambio conserva el aviso, pero evita que esa advertencia específica impida crear fs.img. Se etiquetó como baseline-fedora44 para distinguirlo de las modificaciones funcionales.

```
make clean
make
make qemu CPUS=1
```

Comandos principales de compilación y ejecución.

Las primeras mediciones se realizaron con una sola CPU virtual. Esto reduce la variación causada por ejecución simultánea y permite observar con claridad el orden de selección. Después se repitieron pruebas con las tres CPU virtuales predeterminadas del Makefile para detectar problemas de locks o selección concurrente.

5. Análisis del scheduler original

5.1 Estructuras principales

Elemento	Función en la planificación
enum procstate	Define UNUSED, USED, SLEEPING, RUNNABLE, RUNNING y ZOMBIE.
struct context	Guarda ra, sp y registros s0–s11 para cambios de contexto en el kernel.
struct cpu	Mantiene el proceso actual, el contexto del scheduler y el estado de interrupciones.
struct proc	Contiene lock, state, chan, pid, context, pagetable, trapframe y demás recursos del proceso.
proc[NPROC]	Tabla global fija de hasta 64 procesos.
cpus[NCPU]	Arreglo para un máximo de 8 CPU.
ticks / tickslock	Reloj global incrementado por la CPU 0 y protegido por spinlock.

El scheduler únicamente selecciona entradas en estado RUNNABLE. Cada entrada posee su propio `p->lock`; al cambiarla a RUNNING, el scheduler mantiene ese lock a través de `swtch()`. El proceso liberará el lock al iniciar o lo recuperará antes de regresar al scheduler. `sched()` verifica que el proceso ya no figure como RUNNING, que solo se mantenga el lock esperado y que las interrupciones estén deshabilitadas.

5.2 Flujo de selección

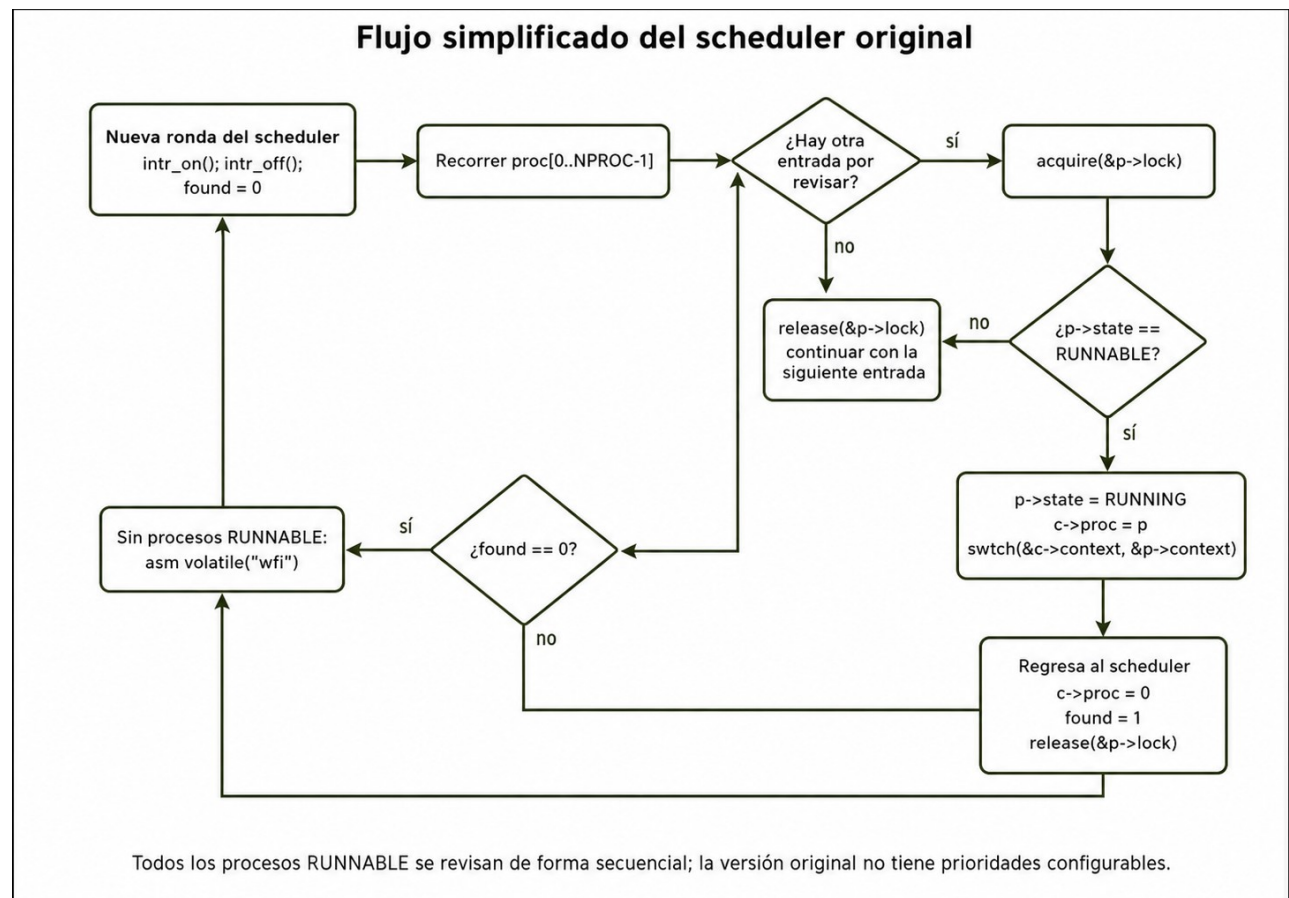


Figura 1. Flujo simplificado del scheduler original de xv6.

En cada ronda, scheduler() habilita y vuelve a deshabilitar interrupciones para evitar deadlocks y carreras con wfi. Luego recorre proc[] desde el inicio. Para cada proceso adquiere p->lock; si está RUNNABLE lo cambia a RUNNING, asigna c->proc y ejecuta swtch(). Cuando el proceso cede la CPU, el recorrido continúa. Si no se encontró ningún proceso listo, la CPU ejecuta wfi hasta una interrupción.

```
for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->state == RUNNABLE) {
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
    }
    release(&p->lock);
}
```

Fragmento representativo de scheduler() en baseline-fedora44.

5.3 Preemptión y cambios de estado

devintr() identifica la interrupción del temporizador y devuelve 2. usertrap() y kerneltrap() comprueban ese valor y llaman a yield() cuando existe un proceso actual. yield() adquiere p->lock, cambia RUNNING por RUNNABLE y llama a sched(). De esta forma, una carga intensiva no conserva la CPU indefinidamente.

sleep() cambia el proceso a SLEEPING y asocia un canal de espera. wakeup() recorre la tabla y devuelve a RUNNABLE los procesos cuyo canal coincide. kkill() también despierta un proceso dormido para permitir que observe la marca killed. Estas transiciones fueron relevantes porque el aging debe contar únicamente el tiempo durante el cual un proceso está listo para ejecutar.

5.4 Comportamiento y limitaciones

El comportamiento puede describirse como un round-robin aproximado basado en el recorrido de la tabla: todos los RUNNABLE reciben oportunidades, pero el orden depende de su posición en proc[]. No hay prioridades configurables ni una noción explícita de antigüedad. La simplicidad reduce el costo y facilita el razonamiento, aunque no permite expresar que una tarea sea más urgente que otra.

Ventajas	Limitaciones
Implementación compacta y fácil de seguir.	No existe prioridad por proceso.
Locks por proceso compatibles con múltiples CPU.	El orden depende del recorrido de proc[].
Preemptión periódica por temporizador.	No ofrece protección explícita contra starvation en una política prioritaria.
Costo lineal en una tabla acotada.	No distingue cargas interactivas, de sistema o de fondo.

6. Análisis de la memoria original

6.1 Sv39 y direcciones virtuales

xv6-riscv utiliza el esquema Sv39. Una dirección virtual se divide en tres índices de 9 bits y un desplazamiento de 12 bits. Cada nivel contiene 512 entradas de 64 bits; el recorrido comienza en la tabla raíz indicada por satp y termina en un PTE hoja que referencia una página física. En xv6, PGSIZE es 4096 bytes.

Dirección virtual Sv39



Cada campo VPN tiene 9 bits, por lo que cada tabla contiene $2^9 = 512$ entradas.



Cada tabla contiene 512 PTE de 64 bits. xv6 usa páginas base de 4096 bytes.

Los bits V, R, W, X y U controlan validez y permisos; los bits RSW están reservados para software y pueden almacenar metadatos, como PTE_COW.

Figura 2. Descomposición simplificada de una dirección Sv39 y recorrido de tablas.

Bit / campo	Significado
PTE_V	Entrada válida.
PTE_R	Lectura permitida.
PTE_W	Escritura permitida.
PTE_X	Ejecución permitida.
PTE_U	Acceso permitido desde modo usuario.
RSW (bits 9–8)	Campo reservado para software del supervisor; el proyecto usa el bit 8 como PTE_COW.

6.2 Funciones de traducción y mapeo

Función	Responsabilidad
walk()	Recorre los tres niveles y, si alloc != 0, crea tablas intermedias.
walkaddr()	Obtiene la dirección física de una página de usuario válida.
mappages()	Crea PTE hoja para un rango alineado de direcciones.
uvmunmap()	Elimina mapeos y opcionalmente libera páginas físicas.
uvmalloc()	Asigna páginas, las pone en cero y crea mapeos de usuario.
uvmdealloc()	Reduce el tamaño y desmonta páginas que dejan de pertenecer al proceso.
uvmfree()	Libera páginas de usuario y las tablas de páginas.
copyin/copyout	Transfiere datos entre kernel y memoria virtual del proceso.

6.3 Asignador físico

kalloc.c administra páginas completas mediante una lista enlazada de estructuras run. kinit() incorpora el rango utilizable a la lista; kalloc() retira la primera página; kfree() la rellena con un patrón para detectar referencias colgantes y la devuelve al freelist. El lock kmem.lock protege la lista en entornos multiprocesador.

6.4 Crecimiento y lazy allocation

La versión base utilizada ya ofrece asignación eager y lazy. sys_sbrk() recibe un tipo: con SBRK_EAGER o reducción de tamaño invoca growproc(); en modo lazy incrementa p->sz sin asignar memoria física. Cuando el proceso accede a una dirección válida todavía no mapeada, usertrap(), copyin() o copyout() llaman a vmfault(), que reserva una página, la inicializa en cero y la mapea con permisos de usuario.

Decisión de diseño: como lazy allocation ya formaba parte de la línea base y además estaba cubierto por usertests, la modificación de memoria se cambió a Copy-on-Write para evitar presentar como propia una funcionalidad existente.

6.5 fork y uvmcopy() original

kfork() crea el descriptor del hijo y llama a uvmcopy() con el tamaño virtual del padre. La implementación original recorre cada página residente, reserva una nueva página con kalloc(), copia 4096 bytes mediante memmove() y mapea la copia en la tabla del hijo. Los huecos de lazy allocation se omiten.

```
if ((mem = kalloc()) == 0)
    goto err;
memmove(mem, (char *)pa, PGSIZE);
if (mappages(new, i, PGSIZE, (uint64)mem, flags) != 0) {
    kfree(mem);
    goto err;
}
```

Núcleo de uvmcopy() en la línea base.

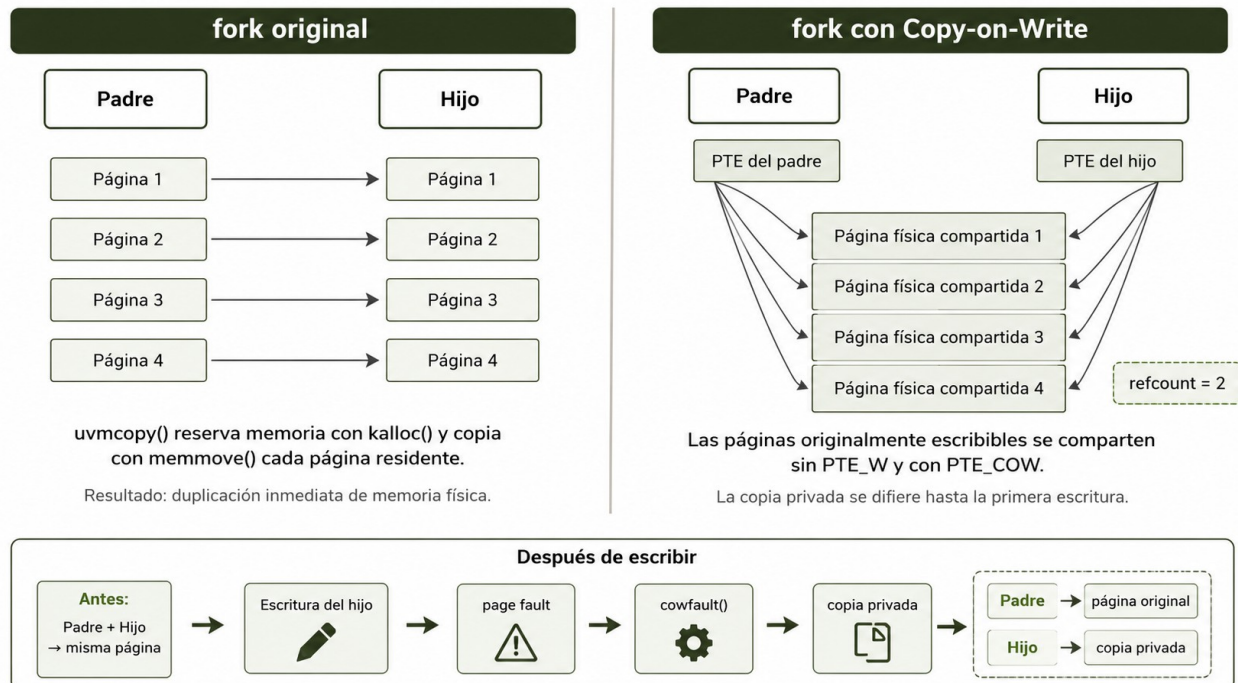


Figura 3. Diferencia conceptual entre la copia inmediata y el mapeo compartido COW.

La copia eager ofrece aislamiento inmediato y simplifica el manejo posterior, pero puede realizar trabajo que nunca será aprovechado. El caso típico es fork seguido de exec: el hijo reemplaza casi toda su memoria y las páginas copiadas se descartan poco después.

7. Diseño del scheduler por prioridades con aging

7.1 Reglas de prioridad

Parámetro	Valor
Prioridad más alta	0
Prioridad más baja	4
Prioridad predeterminada	2
Intervalo de aging	20 ticks
Herencia	El hijo recibe la prioridad base del padre
Tiempo contabilizado	Solo espera continua en RUNNABLE

Cada proceso almacena `priority` y `ready_since`. `priority` es estable hasta que un programa invoca `setpriority()`; `ready_since` identifica el comienzo de una espera continua en RUNNABLE. No se guarda un campo `effective_priority`: el valor se calcula durante la selección, lo que evita actualizaciones periódicas sobre toda la tabla.

Fórmula: $\text{prioridad efectiva} = \max(0, \text{prioridad base} - \lfloor (\text{ticks} - \text{ready_since}) / 20 \rfloor)$.

Ejemplo: un proceso con prioridad base 4 y 45 ticks de espera acumula dos pasos de aging. Su prioridad efectiva es $4 - 2 = 2$. La base continúa siendo 4; al comenzar una nueva espera, `ready_since` se reinicia y el cálculo parte nuevamente de esa base.

7.2 Prevención de starvation

Sin aging, una tarea de prioridad 4 podría no ejecutar mientras exista trabajo constante en prioridad 0. Con el intervalo de 20 ticks, debe esperar aproximadamente 80 ticks para alcanzar prioridad efectiva 0. En ese momento compite dentro del nivel más alto disponible y obtiene CPU aunque la carga original de prioridad 0 siga activa.

7.3 Selección y locking

Scheduler por prioridades con aging

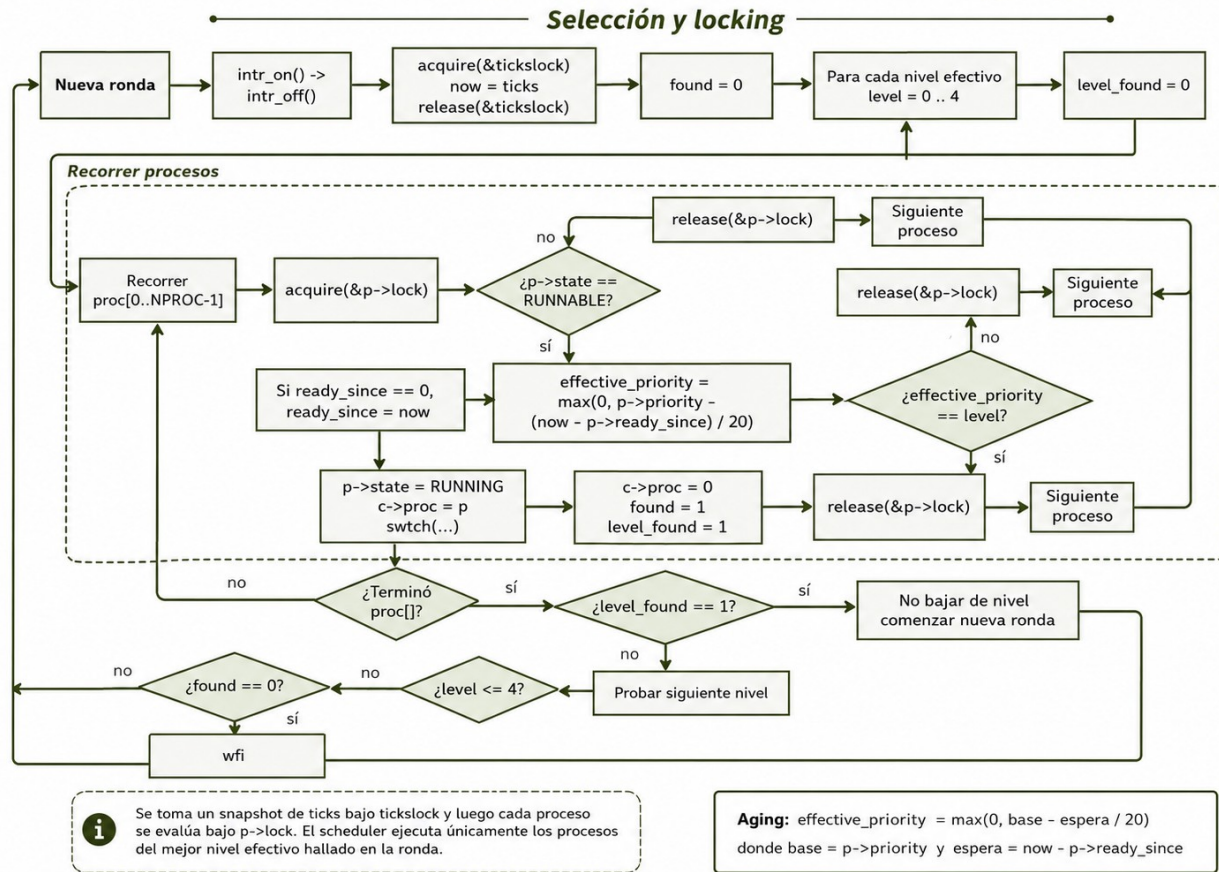


Figura 4. Flujo del scheduler modificado.

La implementación toma una instantánea de ticks bajo tickslock y libera ese lock antes de adquirir cualquier p->lock. Esta separación evita invertir el orden de locks respecto de caminos como sleep(&ticks, &tickslock), que pueden adquirir el lock del proceso mientras todavía mantienen tickslock.

Después se recorren los niveles efectivos de 0 a 4. Para cada nivel se inspecciona $\text{proc}[]$. Si un proceso **RUNNABLE** coincide, se ejecuta de inmediato. Al finalizar el recorrido de ese nivel, si hubo al menos un proceso, no se consideran niveles inferiores durante esa ronda. Con cinco niveles fijos y $\text{NPROC} = 64$, el peor caso inspecciona 320 entradas; asintóticamente continúa siendo $O(\text{NPROC})$ porque el número de niveles es constante.

7.4 Interfaz de usuario

La llamada `setpriority(int)` permite que el proceso actual establezca su prioridad base entre 0 y 4. Los valores externos al rango devuelven `-1`. La syscall se registró en `syscall.h`, `syscall.c`, `sysproc.c`, `usys.pl` y `user.h`.

```
if (priority < PRIORITY_MIN || priority > PRIORITY_MAX)
    return -1;

acquire(&p->lock);
p->priority = priority;
release(&p->lock);
```

Validación central de `sys_setpriority()`.

8. Implementación del scheduler

Archivo	Cambio principal
kernel/param.h	PRIORITY_MIN, PRIORITY_MAX, DEFAULT_PRIORITY y AGING_INTERVAL.
kernel/proc.h	Campos priority y ready_since en struct proc.
kernel/proc.c	Inicialización, herencia, reinicio en transiciones y política de selección.
kernel/syscall.h / syscall.c	Número y registro de SYS_setpriority.
kernel/sysproc.c	Validación y actualización protegida de la prioridad.
user/usys.pl / user.h	Stub y prototipo de usuario.
user/schedtest.c	Carga sincronizada con prioridades 0, 2 y 4.
user/agingtest.c	Carga alta de 120 ticks y proceso bajo que reanuda por aging.
Makefile	Inclusión de programas de prueba.

8.1 Ciclo de vida de los metadatos

allocproc() inicializa prioridad 2 y ready_since 0. freeproc() restablece ambos campos. userinit(), kfork(), yield(), wakeup() y kkill() marcan ready_since = 0 al entrar en RUNNABLE; el scheduler interpreta ese cero como el inicio de una nueva espera y lo reemplaza por la instantánea actual de ticks. kfork() además copia la prioridad base del padre.

```
uint waited = now - p->ready_since;
int aging_steps = waited / AGING_INTERVAL;
int effective_priority = p->priority - aging_steps;
if (effective_priority < PRIORITY_MIN)
    effective_priority = PRIORITY_MIN;
```

Cálculo de prioridad efectiva en scheduler().

8.2 Equidad dentro del nivel

El scheduler recorre toda la tabla para el mejor nivel activo, por lo que varios procesos con la misma prioridad efectiva reciben un turno por ronda siguiendo el orden de proc[]. Después vuelve a comenzar en el nivel 0. Esta política combina prioridad estricta entre niveles y un comportamiento similar a round-robin dentro de cada nivel, sin introducir colas adicionales.

9. Pruebas y resultados del scheduler

9.1 Línea base

Con baseline-scheduler-test, una CPU y tres cargas equivalentes, Child 1, Child 2 y Child 3 finalizaron en ese orden en tres repeticiones. Como no existían prioridades, el resultado fue consistente con el recorrido secuencial de proc[]. Esta prueba estableció una referencia, no una garantía general de orden.

Ejecución baseline	Orden de finalización
1	Child 1 → Child 2 → Child 3
2	Child 1 → Child 2 → Child 3
3	Child 1 → Child 2 → Child 3

9.2 Prueba de prioridades

schedtest crea tres procesos, configura prioridades 0, 2 y 4, y utiliza dos pipes: uno confirma que todos configuraron su prioridad y otro los libera aproximadamente al mismo tiempo. Cada proceso ejecuta 60.000.000 iteraciones y registra ticks de inicio y fin.

Proceso	Prioridad	Tick de finalización	Tiempo observado
Child 1	0	9321	5 ticks desde su inicio
Child 2	2	9326	11 ticks desde su inicio
Child 3	4	9331	5 ticks desde su inicio; comenzó al terminar los superiores

Child 2 imprimió “started” antes que Child 1 en algunas ejecuciones porque la salida de consola y el despertar desde el pipe no son simultáneos. El criterio relevante fue la finalización: prioridad 0 terminó primero, seguida por 2 y luego 4.

9.3 Prueba de aging

agingtest mantiene un proceso de prioridad 0 activo durante 120 ticks. El proceso de prioridad 4 ejecuta inicialmente, duerme un tick y vuelve a RUNNABLE mientras continúa la carga alta. De esta manera se mide su reanudación, no su primera ejecución.

Evento	Tick
Proceso de prioridad 0 inicia	9387
Proceso de prioridad 4 entra	9388
Proceso de prioridad 4 reanuda	9469
Espera observada	81 ticks
Proceso de prioridad 0 termina	9507

El valor esperado era aproximadamente $4 \times 20 = 80$ ticks. La reanudación ocurrió después de 81 ticks y antes de que finalizara el proceso de prioridad alta. La diferencia de un tick es compatible con la granularidad del reloj y con el momento exacto en el que el proceso volvió a RUNNABLE.

9.4 Multiprocesador

Con las tres CPU virtuales predeterminadas, schedtest terminó sin panic, deadlock ni errores de sched. Los mensajes se mezclaron en la consola porque varios harts escribieron al mismo tiempo; esta superposición es una propiedad de la salida concurrente, no una corrupción de los procesos.

10. Diseño de Copy-on-Write

10.1 Objetivo

La meta fue evitar que fork copie inmediatamente cada página física residente. Padre e hijo deben compartirla hasta que alguno intente escribir. Para mantener aislamiento, las páginas compartidas se mapean sin PTE_W y con PTE_COW. Una escritura genera un store page fault, que crea una copia privada solo cuando es necesaria.

10.2 Bit COW y referencias

Se definió PTE_COW como $1L \ll 8$, dentro del campo RSW de los PTE. Además, kalloc.c incorporó un arreglo refcount indexado por dirección física / PGSIZE. kalloc() asigna contador 1; kaddref() lo incrementa; kfree() lo reduce y devuelve la página al freelist únicamente al llegar a cero; kgetref() permite decidir si una copia es realmente necesaria.

10.3 Resolución del fallo

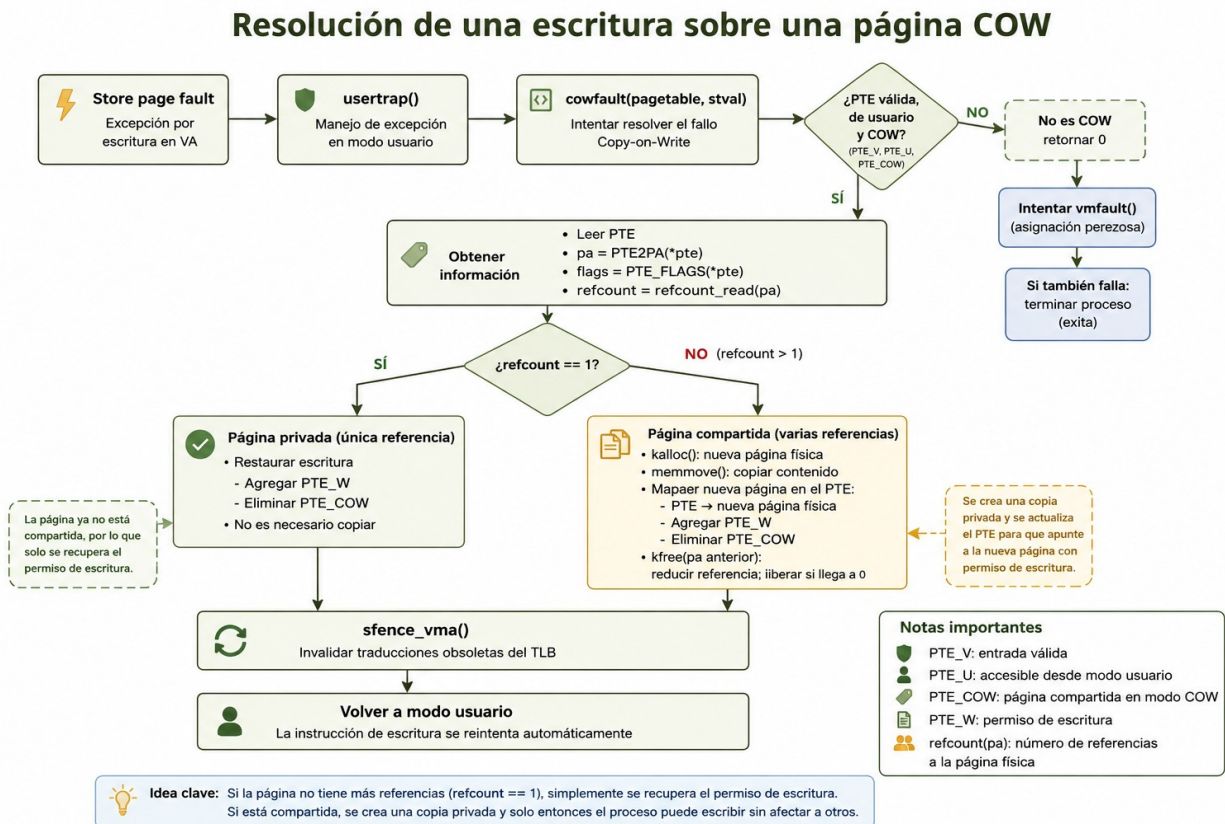


Figura 5. Flujo de cowfault() para una escritura sobre una página compartida.

Si el contador es 1, el proceso ya posee la única referencia: se elimina PTE_COW, se restaura PTE_W y no se copia. Si el contador es mayor que 1, se reserva una página, se copia el contenido, se sustituye el PTE del proceso, se reduce la referencia anterior y se ejecuta sfence_vma() para invalidar traducciones obsoletas del TLB.

10.4 Interacción con lazy allocation y copyout

uvmcopy() continúa omitiendo páginas sin PTE o sin PTE_V, por lo que los huecos lazy permanecen diferidos en padre e hijo. En usertrap(), un store page fault intenta primero cowfault(); si la página no era

COW, se intenta `vmfault()` para lazy allocation. `copyout()` también detecta `PTE_COW` antes de escribir desde el kernel, lo que cubre operaciones como `read()` hacia un buffer heredado después de `fork`.

11. Implementación de Copy-on-Write

Archivo	Cambio principal
<code>kernel/riscv.h</code>	Definición <code>PTE_COW</code> en el bit 8.
<code>kernel/kalloc.c</code>	<code>refcount</code> , inicialización, <code>kaddr()</code> , <code>kgetref()</code> y <code>kfree()</code> condicionado.
<code>kernel/defs.h</code>	Prototipos de auxiliares y <code>cowfault()</code> .
<code>kernel/vm.c</code>	<code>uvmcopy()</code> compartido, <code>cowfault()</code> y soporte en <code>copyout()</code> .
<code>kernel/trap.c</code>	Resolución de store page faults COW antes de lazy allocation.
<code>user/cowtest.c</code>	Prueba con padre, hijo, nieto y 16 páginas.
<code>Makefile</code>	Inclusión de <code>cowtest</code> .

11.1 Nueva `uvmcopy()`

```
if (flags & PTE_W) {
    flags = (flags & ~PTE_W) | PTE_COW;
    *pte = PA2PTE(pa) | flags;
    sfence_vma();
}
if (mappages(new, i, PGSIZE, pa, flags) != 0)
    goto err;
kaddr((void *)pa);
```

Las páginas de escritura se convierten en mapeos compartidos COW.

Las páginas que ya eran genuinamente de solo lectura, como el texto ejecutable, se comparten sin añadir `PTE_COW`. Por tanto, una escritura ilegal sobre código no se convierte accidentalmente en una copia modificable.

11.2 Contadores y liberación

Durante `kinit()`, las páginas utilizables reciben temporalmente una referencia antes de pasar por `freerange()`, que llama a `kfree()` y las deja en cero dentro del freelist. Esta estrategia permite reutilizar la misma lógica de liberación durante el arranque. Cada nueva `kalloc()` vuelve a establecer 1.

```
refcount[index]--;
if (refcount[index] > 0) {
    release(&kmem.lock);
    return;
}
/* solo aquí se incorpora la página al freelist */
```

kfree() no libera memoria todavía compartida.

11.3 Validación de direcciones

La primera versión de `cowfault()` redondeaba la dirección y llamaba a `walk()` sin comprobar `MAXVA`. `usertests`, mediante `MAXVAplus`, provocó `panic: walk`. La corrección agregó `if (va >= MAXVA) return 0;` antes de `PGROUNDDOWN()`. Así, el proceso de prueba recibe el fallo esperado sin derribar el kernel.

12. Resultados y comparación de memoria

12.1 Prueba de aislamiento

cowtest reserva 16 páginas y fuerza su materialización escribiendo P en la primera posición de cada una. Después crea un hijo y un nieto. El nieto escribe G en las 16 páginas; el hijo escribe C; el padre conserva P. Cada proceso verifica su contenido antes de terminar.

```
=== Copy-on-Write test ===
Parent initialized 16 pages with P
Grandchild has private copies: OK
Child has private copies: OK
Parent pages remained unchanged: OK
=== Copy-on-Write test passed ===
```

Salida observada de cowtest.

La prueba comprueba contadores mayores que dos, creación de copias privadas, ausencia de liberaciones prematuras y aislamiento. Como hijo y nieto escriben todas las páginas, esta carga es deliberadamente exigente: el ahorro de copias puede disminuir, pero la corrección queda expuesta con claridad.

12.2 Comparación cuantitativa analítica

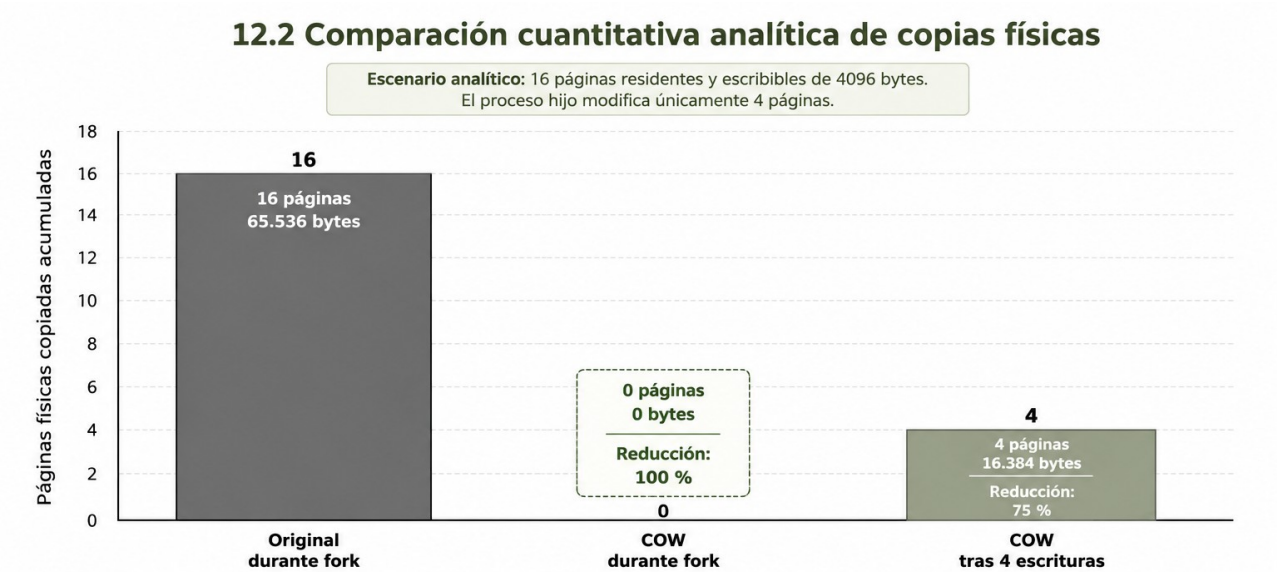


Figura 6. Copias físicas en un escenario analítico de 16 páginas.

Escenario	Páginas copiadas	Bytes copiados	Reducción frente al original
fork original con 16 páginas residentes	16	65.536	0 %
COW inmediatamente después de fork	0	0	100 % inicial
COW si se escriben 4 de 16 páginas	4	16.384	75 %

El cálculo usa páginas de 4096 bytes. Es un escenario de escritura dispersa, común cuando el hijo usa solo parte del espacio heredado o llama a exec(). No se instrumentó un contador de páginas copiadas dentro del kernel; por eso se presenta como análisis teórico derivado del algoritmo. En el cowtest real, hijo y nieto modifican las 16 páginas, de modo que COW puede terminar realizando un volumen de copia similar al original, aunque difiere el momento y evita copias si una página nunca se modifica.

12.3 Comparación cualitativa

Aspecto	fork original	fork con COW
Momento de la copia	Durante fork.	En la primera escritura.
Memoria física inicial	Una página nueva por página residente y por hijo.	Padre e hijo comparten páginas.
Permisos	El hijo recibe permisos equivalentes.	Las páginas modificables quedan sin PTE_W y con PTE_COW.
Fallos posteriores	No requiere fallos por escritura.	Genera store page faults para materializar copias.
Liberación	Cada mapeo posee su página.	Requiere contadores de referencia.
fork + exec	Puede copiar páginas que se descartan.	Evita normalmente esas copias.
Complejidad	Menor.	Mayor: PTE, TLB, refcount y caminos kernel/usuario.

13. Pruebas generales y regresiones

Después de integrar scheduler y COW se ejecutó `usertests -q` con una CPU. La suite recorrió pruebas de `copyin/copyout`, archivos, pipes, procesos, `sbrk`, memoria inválida, lazy allocation, `fork` y límites de direcciones. Algunos casos imprimen `usertrap()`: unexpected scause de manera intencional al intentar accesos prohibidos; el criterio de éxito es que cada caso termine en OK y la suite continúe.

Resultado final: ALL TESTS PASSED.

Prueba	Objetivo	Resultado
forktest	Creación repetida de procesos.	OK
schedtest	Prioridades 0, 2 y 4.	OK
agingtest	Progreso de prioridad baja bajo carga alta.	Reanudó en 81 ticks; OK
cowtest	Aislamiento y refcount con tres procesos.	OK
lazy_alloc / lazy_copy / lazy_sbrk	Compatibilidad con asignación diferida.	OK
MAXVApplus	Direcciones virtuales fuera del rango.	OK después de la validación
usertests -q	Regresión general del sistema.	ALL TESTS PASSED

También se validó el scheduler con tres CPU virtuales. No se observó panic, deadlock, “sched locks” ni “sched RUNNING”. La combinación de pruebas específicas y regresión general reduce el riesgo de que una mejora local haya roto syscalls, memoria o sistema de archivos.

14. Problemas encontrados y depuración

Problema	Diagnóstico	Corrección
GCC 16 detuvo la compilación	-Werror convirtió una advertencia de usertests en error.	-Wno-error=unused-but-set-variable.
Riesgo de inversión de locks	Leer ticks bajo <code>p->lock</code> podía invertir el orden con <code>sleep(&ticks, &tickslock)</code> .	Snapshot de ticks antes de adquirir locks de proceso.
Primera prueba de aging no era válida	El proceso bajo ejecutó casi inmediatamente al salir del pipe.	Dormir un tick y medir el momento de reanudación.
Escritura del kernel sobre COW	<code>copyout()</code> rechazaba páginas sin <code>PTE_W</code> .	Detectar <code>PTE_COW</code> y llamar a <code>cowfault()</code> .
MAXVApplus produjo panic: walk	<code>cowfault()</code> llamó <code>walk()</code> con <code>va >= MAXVA</code> .	Validación previa del rango.
Ediciones con errores de sintaxis	Quedaron fragmentos duplicados o barras sueltas durante reemplazos manuales.	Revisión con <code>nl/sed</code> , compilación incremental y reemplazo completo de funciones cuando fue necesario.
Salida mezclada con varias CPU	<code>printf</code> de procesos concurrentes se intercaló.	Interpretar resultados por terminación y ausencia de panic, no por formato de consola.

La depuración fue parte del método: cada error produjo una hipótesis verificable, una modificación pequeña y una repetición de la prueba que lo reveló. El caso de aging muestra por qué una prueba debe aislar exactamente la propiedad que pretende demostrar; el caso MAXVA muestra la importancia de incluir entradas adversas además de cargas normales.

15. Ventajas y limitaciones

15.1 Scheduler

Ventajas	Limitaciones
Permite expresar importancia mediante cinco niveles.	Número fijo de niveles y prioridad estática.
Aging evita starvation de tareas bajas.	Escanea hasta cinco veces una tabla fija por ronda.
No requiere colas nuevas ni cambios al cambio de contexto.	La equidad dentro del nivel depende del orden de <code>proc[]</code> .

Ventajas	Limitaciones
Funciona con una y varias CPU.	No se midió latencia sobre cargas extensas ni interacción con I/O real.
La syscall facilita pruebas y uso desde programas.	No implementa nice, prioridades por usuario o políticas en tiempo real.

15.2 Copy-on-Write

Ventajas	Limitaciones
Evita copias inmediatas y favorece fork + exec.	Aumenta la complejidad del kernel.
Reduce memoria si pocas páginas se modifican.	Cada escritura inicial puede generar un page fault.
Comparte también páginas de solo lectura sin marcarlas COW.	Requiere refcount correcto en todas las rutas de liberación.
Compatible con lazy allocation y copyout().	Cuando todos escriben todas las páginas, el ahorro disminuye.
Optimiza el caso refcount == 1 sin copiar.	No se añadió instrumentación de rendimiento o memoria en tiempo real.

16. Control de versiones y reproducibilidad

El repositorio público conserva el historial de xv6 y agrega commits pequeños y descriptivos. La rama final es feature/scheduler-priority-aging y el commit de referencia del informe es f83ee36. Mantener el historial original permite comparar cambios y atribuir correctamente el código base.

Tag	Contenido
baseline-original	Fuente oficial antes del ajuste local.
baseline-fedora44	Línea base que compila en Fedora 44 con GCC 16.
scheduler-priority-aging-v1	Scheduler estable con pruebas de prioridades y aging.
cow-fork-v1	Implementación COW estable antes de documentación final.
xv6-riscv-rev5	Tag heredado del código base.

```
git checkout baseline-fedora44
make clean && make
make qemu CPUS=1

git checkout feature/scheduler-priority-aging
make clean && make
make qemu CPUS=1
```

Secuencia de comparación entre línea base y versión final.

La carpeta docs/ incluye notas de análisis, diseño, resultados y evidencias. README.md documenta compilación, pruebas, archivos modificados y tags. Esta separación permite que un evaluador reproduzca el sistema sin depender de la conversación de desarrollo.

17. Demostración

La demostración recomendada dura entre siete y diez minutos y alterna explicación con ejecución. La línea base sirve para presentar la política uniforme; la rama final muestra las diferencias observables.

1. Mostrar el repositorio, la rama final y los tags.
2. Cambiar a baseline-fedora44, compilar y arrancar con CPUS=1.
3. Explicar scheduler() original y ejecutar la prueba baseline.
4. Volver a feature/scheduler-priority-aging y compilar.
5. Ejecutar schedtest y comentar prioridades, ticks y orden de finalización.
6. Ejecutar agingtest y señalar waited=81 antes de la terminación de la carga alta.
7. Explicar uvmcopy() original, PTE_COW y refcount.
8. Ejecutar cowtest y mostrar la independencia P/C/G.
9. Mostrar el resultado ALL TESTS PASSED y cerrar con ventajas y límites.

18. Conclusiones

El análisis del scheduler permitió relacionar el recorrido de `proc[]`, los estados `RUNNABLE` y `RUNNING`, los locks por proceso, las interrupciones del temporizador y `swtch()`. La modificación mostró que una política de prioridad no se limita a escoger “el número menor”: también debe registrar correctamente el tiempo de espera, conservar el orden de locks y definir qué ocurre cuando varios procesos comparten nivel.

El aging resolvió el riesgo de starvation de forma verificable. Un proceso de prioridad 4 reanudó tras 81 ticks, prácticamente los 80 calculados, mientras una carga de prioridad 0 seguía activa. La prueba fue relevante porque midió una reanudación después de entrar nuevamente en `RUNNABLE`, corrigiendo una primera versión que no aislaba la propiedad.

En memoria, el recorrido por `Sv39`, `PTE`, `kalloc()`, `uvmmalloc()`, `vmfault()` y `uvmcopy()` permitió distinguir asignación virtual, asignación física y permisos. Copy-on-Write trasladó el costo de copiar desde `fork` hasta la primera escritura. El uso de `PTE_COW`, contadores de referencia, `cowfault()` y soporte en `copyout()` mantuvo el aislamiento incluso con padre, hijo y nieto compartiendo 16 páginas.

La suite general fue esencial: reveló el caso MAXVA que las pruebas normales no habían expuesto y confirmó después que scheduler, COW, lazy allocation, archivos y procesos seguían funcionando juntos. El resultado final cumple los objetivos del proyecto: dos modificaciones reales, análisis de la línea base, comparación antes/después, código comentado, control de versiones, documentación reproducible y demostración preparada.

19. Referencias

Enlaces consultados

[Curso MIT 6.1810 y materiales de xv6](#)

[Repositorio oficial xv6-riscv](#)

[Especificación RISC-V Supervisor ISA](#)

[Repositorio público del proyecto](#)

Anexo A. Archivos modificados

Subsistema	Archivos
Compatibilidad	Makefile
Scheduler	kernel/param.h, kernel/proc.h, kernel/proc.c
Syscall de prioridad	kernel/syscall.h, kernel/syscall.c, kernel/sysproc.c, user/usys.pl, user/user.h
Copy-on-Write	kernel/riscv.h, kernel/kalloc.c, kernel/defs.h, kernel/vm.c, kernel/trap.c
Pruebas	user/schedtest.c, user/agingtest.c, user/cowtest.c
Documentación	README.md y docs/

Comandos de validación final

```
make clean
make
echo $?
make qemu CPUS=1

# dentro de xv6
usertests -q
cowtest
schedtest
agingtest
```

La compilación terminó con código 0 y usertests finalizó con ALL TESTS PASSED.